

Nuestra estrategia

Vamos a construirlo en este orden:

Especificación → Arquitectura → Base de datos → Diseño UI → Implementación → Pruebas → Refinamiento

Y pondría una regla fundamental:

La IA no puede programar una funcionalidad que no esté definida en la especificación.

MVP — Nombre provisional

Todavía no necesitamos decidir el nombre comercial.

Concepto

Una aplicación móvil de desarrollo personal gamificada que ayuda al usuario a:

 **definir sueños** →  **convertirlos en metas** →  **realizar misiones** → 
desarrollar disciplina →  **conseguir resultados.**

La gamificación existe para incentivar acciones reales, no para mantener al usuario pegado a la pantalla.

STACK DEL MVP

Para la primera versión:

Frontend

Flutter + Dart

Backend

Supabase

Base de datos

PostgreSQL

Autenticación

Supabase Auth

Notificaciones

Firestore Cloud Messaging

IA ❌ NO en MVP

Pagos ❌ NO en la primera versión funcional

Primero validamos el producto.



MÓDULOS DEL MVP

Vamos a congelar inicialmente estos:

01 — Onboarding

Presentar el concepto y crear el perfil.

02 — Sueños y metas

El usuario define aquello que quiere conseguir.

03 — Misiones

Acciones diarias relacionadas con sus objetivos.

04 — Sistema de progreso

⚡ Impulso, niveles y progreso.

05 — Rachas

🔥 Constancia diaria.

06 — Chispas

✨ Moneda/recompensa.

07 — Logros

🏆 Hitos desbloqueables.

08 — Perfil

Nivel, disciplina, estadísticas y logros.

09 — Notificaciones

Motivación, recordatorios y mensajes de apoyo.

Una decisión importante

Para el MVP **no dejaría que la IA genere las misiones.**

Tendríamos un catálogo estructurado de misiones.

Por ejemplo:

Misión

- título
- descripción
- categoría
- dificultad
- duración
- impulso
- chispas
- tipo

Categorías:

-  Mente
-  Cuerpo
-  Relaciones
-  Crecimiento
-  Finanzas
-  Creatividad
-  Experiencias
-  Bienestar

Posteriormente la IA podrá utilizar estos datos para crear un sistema muchísimo más personalizado.

REGLAS DEL JUEGO

Tenemos que definir las **antes de programar**.

Impulso

Es el progreso principal.

Ejemplo:

Dificultad	Impulso
Fácil	10
Normal	25
Difícil	50
Épica	100

Chispas

Moneda interna.

Dificultad	Chispas
Fácil	5
Normal	10
Difícil	20
Épica	50

Racha

Una racha aumenta cuando el usuario cumple su objetivo diario.

Ejemplo:

Día 1 →  1

Día 7 → 🔥 7

Día 30 → 🔥 30

Pero quiero conservar nuestra filosofía:

Perder un día no significa perder todo el progreso.

Podemos tener posteriormente protectores de racha.

NIVELES

El nivel se calcula mediante el **Impulso acumulado**.

No vamos a guardar manualmente "nivel 8" como una verdad independiente.

La fuente de verdad será:

Impulso total → cálculo del nivel

Esto es importante desde el punto de vista técnico porque evita inconsistencias.

LOGROS

Los logros deben ser **determinísticos**.

Por ejemplo:

FIRST_MISSION

Completa tu primera misión.

SEVEN_DAY_STREAK

Consigue una racha de 7 días.

TEN_MISSIONS

Completa 10 misiones.

LEVEL_10

Alcanza el nivel 10.

La aplicación puede detectar automáticamente cuándo se cumple cada condición.



Aquí está la parte que diferencia nuestro producto.

Una meta tendrá:

Meta

- título
- descripción
- categoría
- fecha objetivo
- estado
- progreso
- misiones relacionadas

Ejemplo:

 **Viajar a Japón**

Progreso: **37%**

24 misiones completadas

 12 días de constancia



PANTALLA PRINCIPAL

La Home debería comunicar inmediatamente:

Buenos días, Carlos 

 12 días de racha

NIVEL 8



820 / 1.000 

 MISIÓN DE HOY

Camina durante 20 minutos

+25 

+10 

[COMPLETAR]

☀ MIS METAS

Aprender inglés	68%
Crear mi negocio	34%

🏆 Próximo logro
"Constante"

Faltan 3 misiones

No necesitamos meter 50 elementos.

La pantalla tiene que responder:

¿Qué debo hacer hoy y cuánto estoy avanzando hacia lo que quiero?



BASE DE DATOS

Desde el principio la diseñaría con entidades separadas:

profiles
goals
missions
daily_missions
mission_completions
achievements
user_achievements
rewards
user_rewards
streaks
notifications

Y algo fundamental:

mission_completions

No simplemente:

mission.completed = true

Porque necesitamos historial.

Por ejemplo:

Usuario

Misión

Fecha

Hora

Recompensa obtenida

Así posteriormente podemos construir estadísticas, IA, análisis de comportamiento, etc.

SEGURIDAD

Desde el MVP:

- Autenticación segura.
- Row Level Security de Supabase.
- Un usuario solamente puede acceder a sus propios datos.
- Las recompensas no deben poder modificarse desde el cliente.
- Las operaciones importantes deben validarse en backend.
- Nunca almacenar secretos/API keys dentro de Flutter.
- Base de datos preparada para auditoría de recompensas.

Esto último es **muy importante** porque tenemos una economía interna.

No queremos que alguien pueda modificar:

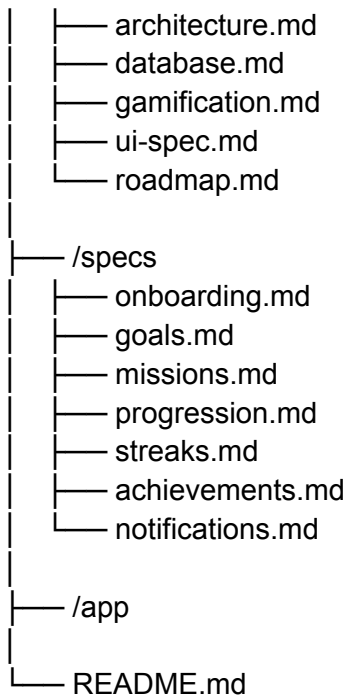
```
chispas = 999999
```

desde la aplicación.

Y ahora viene el SDD

Yo crearía un repositorio con esta estructura:

```
/project
|
|— /docs
|   |— product-spec.md
```



La IA deberá leer esos documentos **antes de modificar código**.



PROMPT MAESTRO PARA TU IA DE CÓDIGO

Este sería el primer prompt que le daría.

PROYECTO: Aplicación móvil gamificada de desarrollo personal

METODOLOGÍA

Utiliza Spec-Driven Development (SDD).

NO empieces programando inmediatamente.

Primero debes analizar esta especificación, identificar ambigüedades, proponer la arquitectura y crear la documentación necesaria.

La especificación del producto será la fuente de verdad.

No debes inventar funcionalidades fuera del alcance definido.

Si una decisión importante no está especificada, debes detenerte y proponer opciones antes de implementarla.

1. OBJETIVO DEL PRODUCTO

Construir una aplicación móvil de desarrollo personal gamificada.

La aplicación ayuda al usuario a transformar sus sueños y metas en pequeñas acciones diarias.

Concepto:

SUEÑO → META → MISIÓN → CONSTANCIA → DISCIPLINA → RESULTADO

La gamificación debe motivar al usuario a realizar acciones en la vida real.

No queremos crear simplemente un habit tracker.

2. DIFERENCIACIÓN

El producto no debe posicionarse principalmente como:

- habit tracker
- aplicación de productividad
- juego RPG

La propuesta central es:

"Convierte tus sueños en misiones diarias."

La aplicación debe hacer sentir al usuario que está progresando hacia algo importante para su vida.

3. ALCANCE DEL MVP

El MVP incluye únicamente:

1. Onboarding
2. Perfil

3. Sueños y metas
4. Misiones
5. Sistema de Impulso
6. Sistema de niveles
7. Rachas
8. Sistema de Chispas
9. Logros
10. Recompensas básicas
11. Notificaciones
12. Estadísticas básicas

NO incluir IA en el MVP.

NO incluir comunidad en el MVP.

NO incluir chat con IA.

NO incluir rankings.

NO incluir temporadas.

NO incluir marketplace.

NO incluir funcionalidades sociales.

NO implementar funcionalidades futuras hasta que estén expresamente aprobadas.

4. STACK

Frontend:

Flutter + Dart

Backend:

Supabase

Database:

PostgreSQL

Authentication:

Supabase Auth

Notifications:

Firebase Cloud Messaging

El código debe estar preparado para incorporar posteriormente un backend adicional y servicios de IA si el producto lo requiere.

5. PRINCIPIOS DE ARQUITECTURA

Utilizar una arquitectura limpia, modular y mantenible.

Separar claramente:

- presentación
- lógica de negocio
- datos
- servicios
- modelos
- navegación
- autenticación

Evitar código monolítico.

Evitar duplicación.

Utilizar componentes reutilizables.

La lógica de gamificación no debe estar mezclada directamente con widgets de Flutter.

Crear servicios independientes para:

- progreso
- niveles
- rachas
- recompensas
- logros
- misiones

6. MODELO DE DATOS

Diseñar inicialmente entidades para:

profiles

goals

missions

daily_missions

mission_completions

achievements

user_achievements

rewards
user_rewards
streaks
notifications

Diseñar las relaciones y restricciones antes de crear las tablas.

Utilizar UUID para identificadores.

Agregar timestamps.

Preparar la base de datos para crecimiento futuro.

Utilizar Row Level Security.

Un usuario nunca debe poder acceder a información privada de otro usuario.

7. GAMIFICACIÓN

IMPULSO

El Impulso es el sistema principal de progreso.

Recompensas iniciales:

Fácil: 10
Normal: 25
Difícil: 50
Épica: 100

CHISPAS

Las Chispas son una moneda interna.

Fácil: 5
Normal: 10
Difícil: 20
Épica: 50

Las Chispas podrán utilizarse posteriormente para desbloqueables.

NIVELES

El nivel debe derivarse del Impulso acumulado.

No crear inconsistencias almacenando manualmente múltiples valores derivados.

Centralizar la lógica de cálculo.

RACHA

Registrar la actividad diaria del usuario.

La racha debe calcularse de forma consistente según las reglas definidas.

No eliminar el progreso histórico si el usuario pierde una racha.

LOGROS

Crear un sistema extensible basado en condiciones.

Ejemplos:

FIRST_MISSION
SEVEN_DAY_STREAK
TEN_MISSIONS
LEVEL_10

8. MISIONES

Las misiones tendrán:

- id
- título
- descripción
- categoría
- dificultad
- duración estimada
- Impulso
- Chispas
- estado

Categorías iniciales:

Mente
Cuerpo
Relaciones
Crecimiento
Finanzas

Creatividad
Experiencias
Bienestar

Durante el MVP las misiones proceden de un catálogo definido previamente.

No utilizar IA para generar misiones.

9. METAS

Una meta debe permitir al usuario definir:

- título
- descripción
- categoría
- fecha objetivo
- estado
- progreso

Una meta puede relacionarse con múltiples misiones.

El sistema debe permitir posteriormente analizar cuánto progreso ha realizado el usuario hacia cada meta.

10. HOME

La pantalla principal debe responder rápidamente:

1. ¿Qué tengo que hacer hoy?
2. ¿Cuál es mi progreso?
3. ¿Cuál es mi racha?
4. ¿Qué tan cerca estoy de mis metas?

Debe mostrar como mínimo:

- saludo
- racha
- nivel
- Impulso
- progreso hacia siguiente nivel
- misiones del día
- metas activas

- próximo logro

La interfaz debe sentirse moderna, limpia y gamificada sin parecer una aplicación infantil.

11. NOTIFICACIONES

Implementar notificaciones para:

- recordatorios de misiones
- mantenimiento de racha
- felicitaciones
- mensajes motivacionales
- mensajes de apoyo
- hábitos saludables

Las notificaciones deben ser configurables.

No enviar mensajes excesivamente frecuentes.

El sistema debe poder ampliarse posteriormente con personalización basada en comportamiento.

12. UX

La aplicación debe ser:

- sencilla
- visual
- rápida
- motivadora
- moderna
- accesible

Evitar sobrecargar las pantallas.

El usuario debe poder completar una misión con muy pocos pasos.

Completar una misión debe generar una experiencia satisfactoria visualmente.

13. SEGURIDAD

Implementar:

- autenticación
- Row Level Security
- validación de datos
- protección de operaciones sensibles
- separación de secretos
- ninguna API key privada dentro de Flutter

Las recompensas y progreso deben validarse de forma segura.

No confiar exclusivamente en valores enviados desde el cliente.

14. PRUEBAS

Antes de considerar una funcionalidad terminada debe existir:

- lógica verificable
- validaciones
- manejo de errores
- pruebas de casos normales
- pruebas de casos límite

Prestar especial atención a:

- cambio de día
 - pérdida de racha
 - completar dos veces una misión
 - recompensas duplicadas
 - cambio de zona horaria
 - creación/eliminación de metas
 - recuperación de sesión
 - pérdida de conexión
-

15. REGLA DE DESARROLLO

Implementar por fases.

FASE 1:
Arquitectura y documentación.

FASE 2:
Base de datos y autenticación.

FASE 3:
Onboarding y perfil.

FASE 4:
Metas.

FASE 5:
Misiones.

FASE 6:
Gamificación.

FASE 7:
Rachas y logros.

FASE 8:
Notificaciones.

FASE 9:
Estadísticas.

FASE 10:
Testing y refinamiento.

No implementar todas las fases de una sola vez.

Después de cada fase:

1. Ejecutar pruebas.
2. Revisar errores.
3. Verificar que cumple la especificación.
4. Actualizar documentación.
5. Esperar aprobación antes de avanzar a una fase arquitectónicamente importante.

16. REGLA FUNDAMENTAL

Antes de escribir código:

1. Analiza la especificación.
2. Propón la arquitectura.

3. Identifica decisiones pendientes.
4. Crea los documentos SDD.
5. Define el modelo de datos.
6. Define los contratos de los servicios.
7. Define los criterios de aceptación.
8. Espera aprobación.

No quiero código apresurado.

Quiero una base sólida, mantenible y preparada para evolucionar.

El objetivo no es simplemente "hacer que funcione".

El objetivo es construir correctamente el núcleo de un producto que posteriormente pueda incorporar IA, suscripciones, comunidad, personalización avanzada y otras funcionalidades sin tener que rehacer toda la arquitectura.

Y haría algo todavía más importante

No le entregaría a la IA este prompt y le diría "construye toda la app".

Ese es precisamente el error que queremos evitar con SDD.

Primero le damos:

Fase 1 → especificación técnica.

Que nos devuelva:

- arquitectura de carpetas
- modelo de datos
- diagrama de relaciones
- navegación
- servicios
- reglas de gamificación
- criterios de aceptación
- estrategia de testing

Nosotros revisamos eso.

Después:

Fase 2 → Supabase + base de datos.

Después:

Fase 3 → aplicación.

De esa manera, si dentro de seis meses queremos meter **IA que analice las metas del usuario, genere misiones personalizadas y funcione como coach**, no tendremos que tirar el MVP y empezar de nuevo.

Y hay otra decisión que debemos tomar **antes de escribir la primera línea de código**: el **sistema exacto de niveles, Impulso, Chispas, rachas y metas**, porque esas reglas son el corazón del producto y conviene dejarlas matemáticamente cerradas en la especificación.